```
array     # => ['string']
duplicate # => ['foo']
```

If the object is not duplicable, `deep_dup` will just return it:

```
number = 1
duplicate = number.deep_dup
number.object_id == duplicate.object_id   # => true
```

Defined in `active_support/core_ext/object/deep_dup.rb`.

2.5 `try`

When you want to call a method on an object only if it is not `nil`, the simplest way to achieve it is with conditional statements, adding unnecessary clutter. The alternative is to use `try`. `try` is like `Object#send` except that it returns `nil` if sent to `nil`.

Here is an example:

```
# without try
unless @number.nil?
  @number.next
end

# with try
@number.try(:next)
```

Another example is this code from `ActiveRecord::ConnectionAdapters::AbstractAdapter`where `@logger` could be `nil`. You can see that the code uses `try` and avoids an unnecessary check.

```
def log_info(sql, name, ms)
  if @logger.try(:debug?)
    name = '%s (%.1fms)' % [name || 'SQL', ms]
    @logger.debug(format_log_entry(name, sql.squeeze(' ')))
  end
end
```

`try` can also be called without arguments but a block, which will only be executed if the object is not nil:

```
@person.try { |p| "#{p.first_name} #{p.last_name}" }
```

Defined in `active_support/core_ext/object/try.rb`.

2.6 `class_eval(*args, &block)`

You can evaluate code in the context of any object's singleton class using `class_eval`:

```
class Proc
  def bind(object)
    block, time = self, Time.current
    object.class_eval do
      method_name = "__bind_#{time.to_i}_#{time.usec}"
      define_method(method_name, &block)
      method = instance_method(method_name)
      remove_method(method_name)
      method
    end.bind(object)
  end
end
```

Defined in `active_support/core_ext/kernel/singleton_class.rb`.

2.7 `acts_like?(duck)`

The method `acts_like?` provides a way to check whether some class acts like some other class based on a simple convention: a class that provides the same interface as `String` defines

```
def acts_like_string?
end
```

which is only a marker, its body or return value are irrelevant. Then, client code can query for duck-type-safeness this way: